
TÍTULO DE LA ACTIVIDAD

Actividad 5

Materia:	Gestión de Proyectos de Inteligencia Artificial
Actividad:	Actividad 5.
Alumno(a):	JUAN ANDRES GUTIERREZ MARTINEZ
Matrícula:	07144441
Docente:	LUIS ARIEL VAZQUEZ PEÑA
Fecha de entrega:	28 / 06 / 2026

Declaración de Uso de Inteligencia Artificial

El alumno debe indicar de manera honesta si utilizó herramientas de Inteligencia Artificial (IA) para el desarrollo de esta actividad. Completa la siguiente tabla:

Herramienta de IA utilizada	¿La usaste?	Propósito / Descripción de uso
ChatGPT / OpenAI	☐ No	
Claude (Anthropic)	☐ Si	Apoyo en la limpieza de datos, estructuración del código (datos_prep.py / train.py), configuración de MLflow y redacción del reporte.
Gemini (Google)	☐ No	
Copilot (Microsoft)	☐ No	
Otra (especifica):	☐ No	

Desarrollo de Contenido

1. Introducción

El presente trabajo integra las practicas de gestion de datos y de MLOps revisadas en el curso, mediante un proyecto aplicado de extremo a extremo: seleccion de un dataset publico, preparacion y versionamiento de los datos, entrenamiento de modelos de aprendizaje automatico, ajuste de hiperparametros y registro de los experimentos en MLflow.

El objetivo es garantizar la trazabilidad, la reproducibilidad y el analisis comparativo de resultados. Se aborda un problema de clasificacion binaria utilizando el dataset Titanic, comparando dos algoritmos (Regresion Logistica y Random Forest) bajo metricas estandarizadas y una validacion estadistica de las diferencias observadas.

2. Desarrollo

2.1 Descripcion del dataset

Se utiliza el dataset Titanic, de dominio publico, obtenido mediante la libreria seaborn (`seaborn.load_dataset('titanic')`). Contiene 891 registros de pasajeros y la variable objetivo 'survived' (1 = sobrevivio, 0 = no sobrevivio). Diccionario de datos de las variables empleadas: pclass (clase del boleto), sex (sexo), age (edad), sibsp (hermanos/conyuge a bordo), parch (padres/hijos a bordo), fare (tarifa), embarked (puerto de embarque) y la variable derivada family_size.

Supuestos: las columnas alive, class, embark_town, adult_male, who y alone se consideran redundantes o derivadas, y deck se descarta por presentar cerca del 77% de valores nulos.

2.2 Limpieza y estandarizacion

El proceso (implementado en `datos_prep.py`) aplica: (1) eliminacion de columnas redundantes o con fuga de informacion -alive es copia exacta de la etiqueta-; (2) regla de negocio que descarta valores imposibles ($\text{fare} < 0$ y $\text{age} \leq 0$); (3) imputacion de nulos: age y fare con la mediana y embarked con la moda; (4) ingenieria de la variable $\text{family_size} = \text{sibsp} + \text{parch} + 1$; (5) codificacion categorica: sex a 0/1 y embarked mediante one-hot; (6) escalado con StandardScaler aplicado dentro del Pipeline de entrenamiento -solo para la Regresion Logistica- para evitar la fuga del conjunto de prueba. El dataset resultante tiene 891 registros, 11 columnas y 0 valores nulos.

2.3 Estrategia de versionamiento

Los datos se versionan en dos carpetas: `datos/datos_ini` (crudo) y `datos/datos_limp` (limpio), manteniendo el original intacto. Los cambios de datos, codigo y experimentos se documentan en `CHANGELOG.md` y `README.md`, y el historial se gestiona con Git/GitHub. Cada experimento queda ademas versionado en MLflow con sus parametros, metricas y artefactos.

2.4 Visualizaciones (patrones y anomalias)

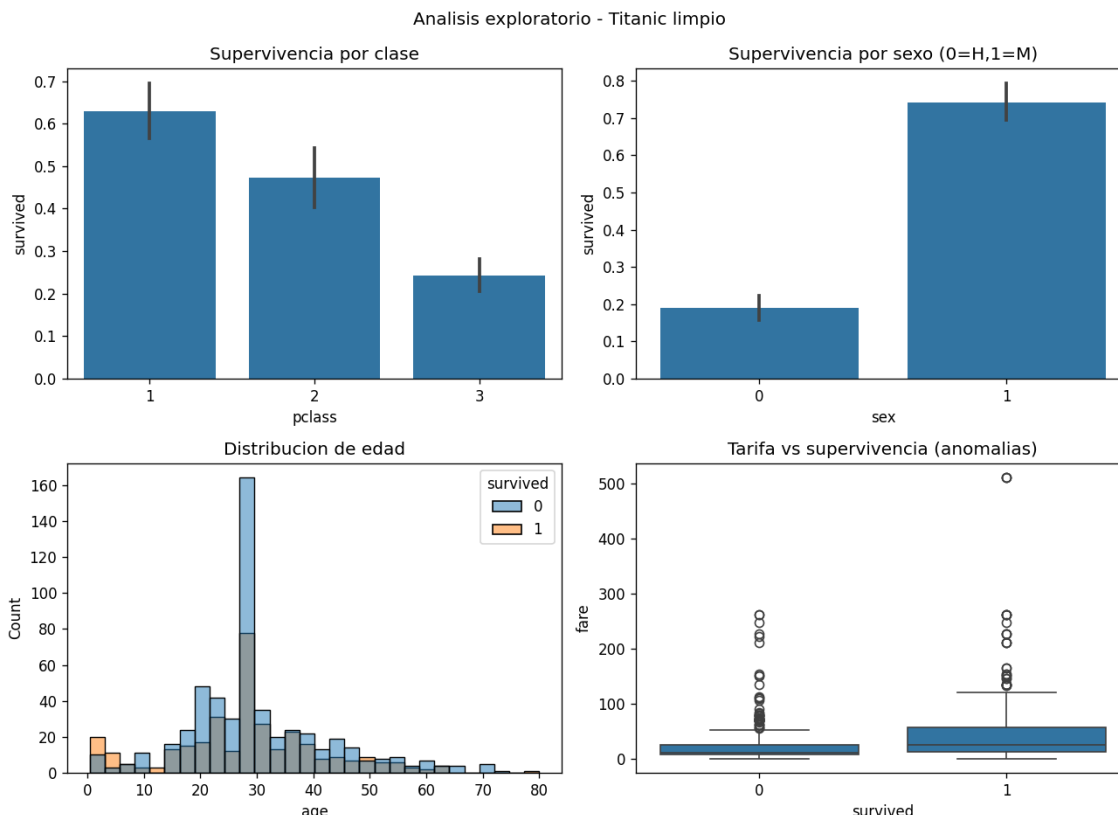


Figura 1. Patrones: la supervivencia aumenta en primera clase y es notablemente mayor en mujeres; el boxplot de tarifa muestra valores atipicos (anomalias).

2.5 Modelos entrenados y configuracion de hiperparametros

Se comparan dos modelos. Regresion Logistica (con StandardScaler), ajustando el parametro de regularizacion C en {0.01, 0.1, 1, 10}. Random Forest, ajustando n_estimators {100, 300}, max_depth {None, 5, 10} y min_samples_split {2, 5}. La seleccion se realiza con GridSearchCV optimizando F1 mediante validacion cruzada estratificada de 5 folds (random_state=42). Mejores configuraciones: Regresion Logistica C=10; Random Forest n_estimators=300, max_depth=10, min_samples_split=2.

2.6 Metricas obtenidas

Modelo	Accuracy	Precision	Recall	F1	ROC-AUC
Regresion Logistica	0.804	0.793	0.667	0.724	0.844
Random Forest	0.804	0.783	0.681	0.729	0.839

Tabla 1. Metricas en el conjunto de prueba (20%).

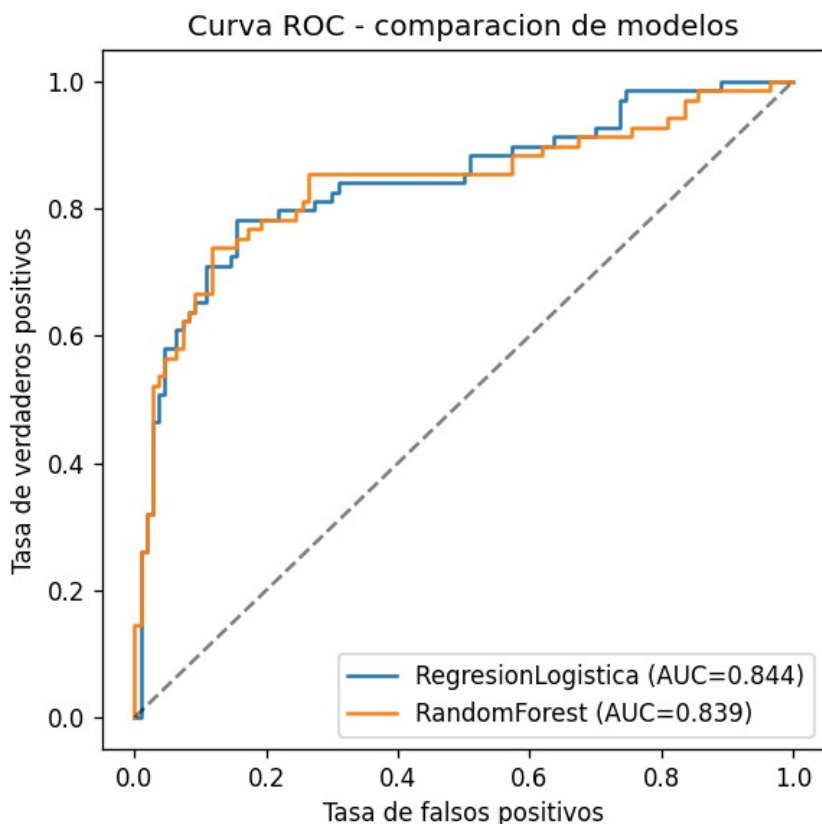


Figura 2. Curva ROC comparativa: ambos modelos presentan un AUC cercano a 0.84.

2.7 Comparacion estadistica de resultados

Para determinar si la diferencia de desempeño es significativa, se compararon los F1 de los 5 folds de validacion cruzada con dos pruebas: la prueba t para muestras emparejadas ($p \approx 0.07$) y la prueba de Wilcoxon ($p \approx 0.13$). En ambos casos $p > 0.05$, por lo que NO existe diferencia estadisticamente significativa entre los modelos: se consideran equivalentes.

2.8 Evidencia del uso de MLflow

Se configuro un servidor MLflow local (sqlite:///mlflow.db, artefactos en ./mlruns) en <http://127.0.0.1:5000>. En el experimento 'Actividad5_Titanic' se registraron tres corridas (RegresionLogistica, RandomForest y Comparacion_estadistica) con sus parametros, metricas y artefactos (modelos serializados y figuras). A continuacion se muestran las capturas del panel.

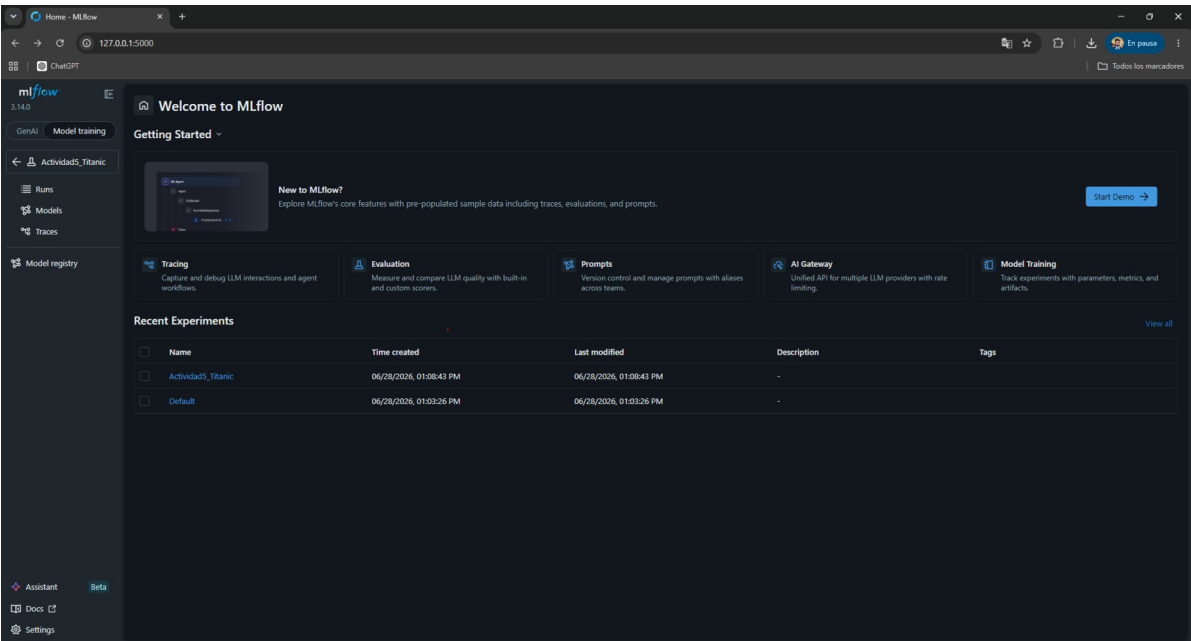


Figura 3. Panel principal de MLflow con el experimento 'Actividad5_Titanic' registrado.

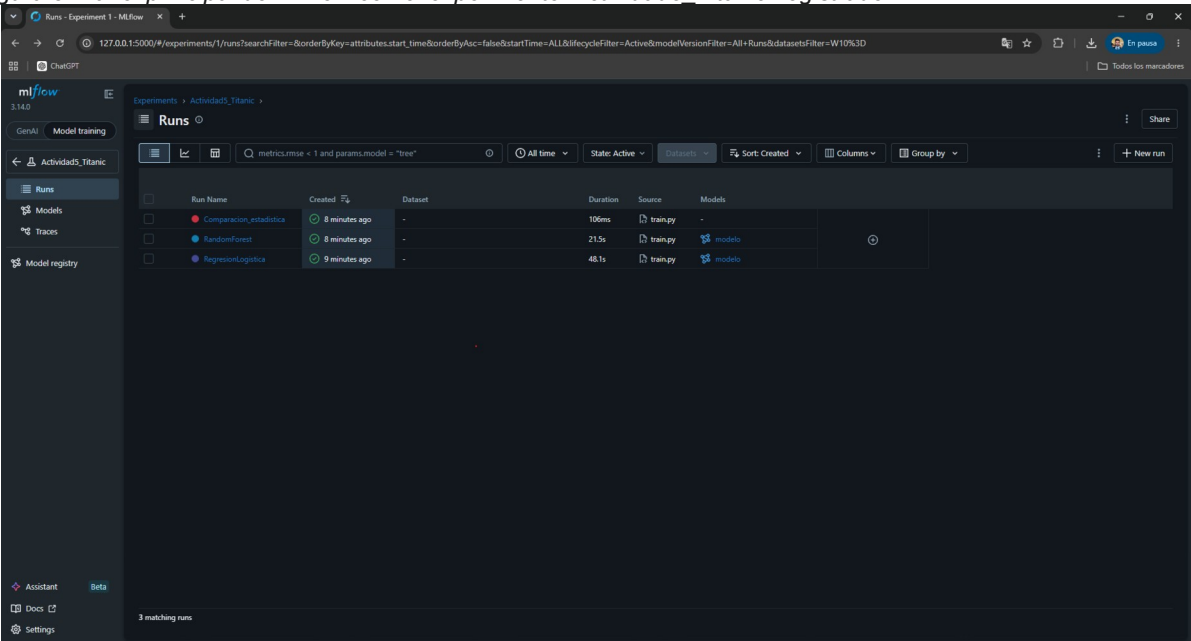


Figura 4. Listado de las 3 corridas del experimento (RegresionLogistica, RandomForest, Comparacion_estadistica), todas finalizadas y con origen train.py.

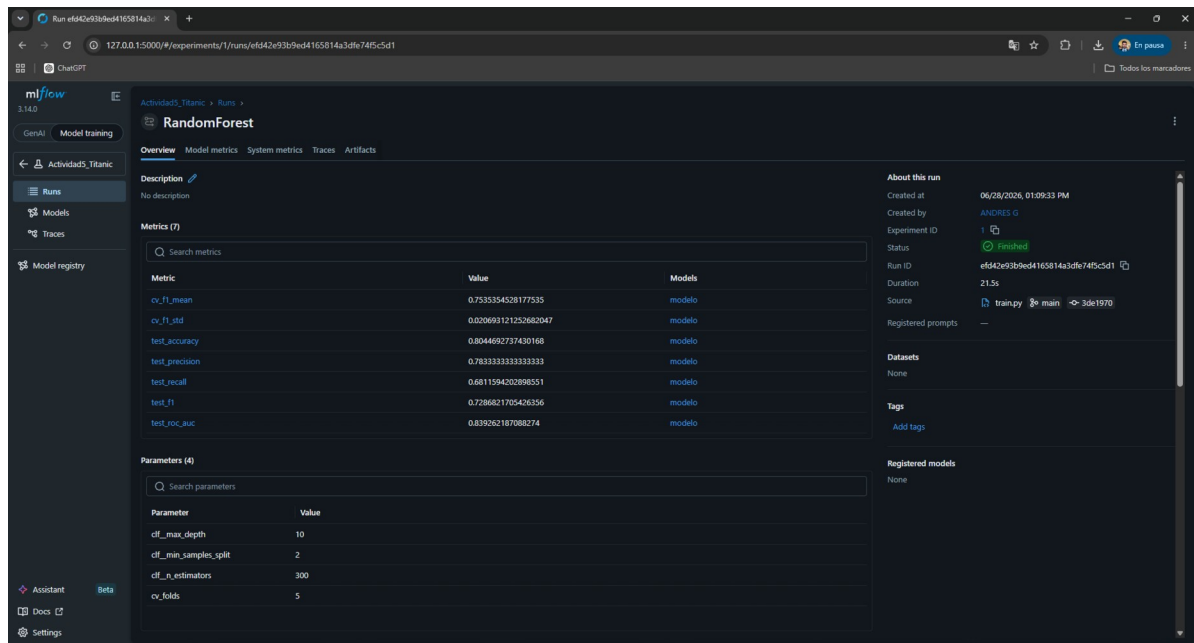


Figura 5. Detalle de la corrida RandomForest: 7 metricas registradas (test_accuracy, test_f1, test_roc_auc, cv_f1_mean, etc.) y 4 parametros (hiperparametros optimos).

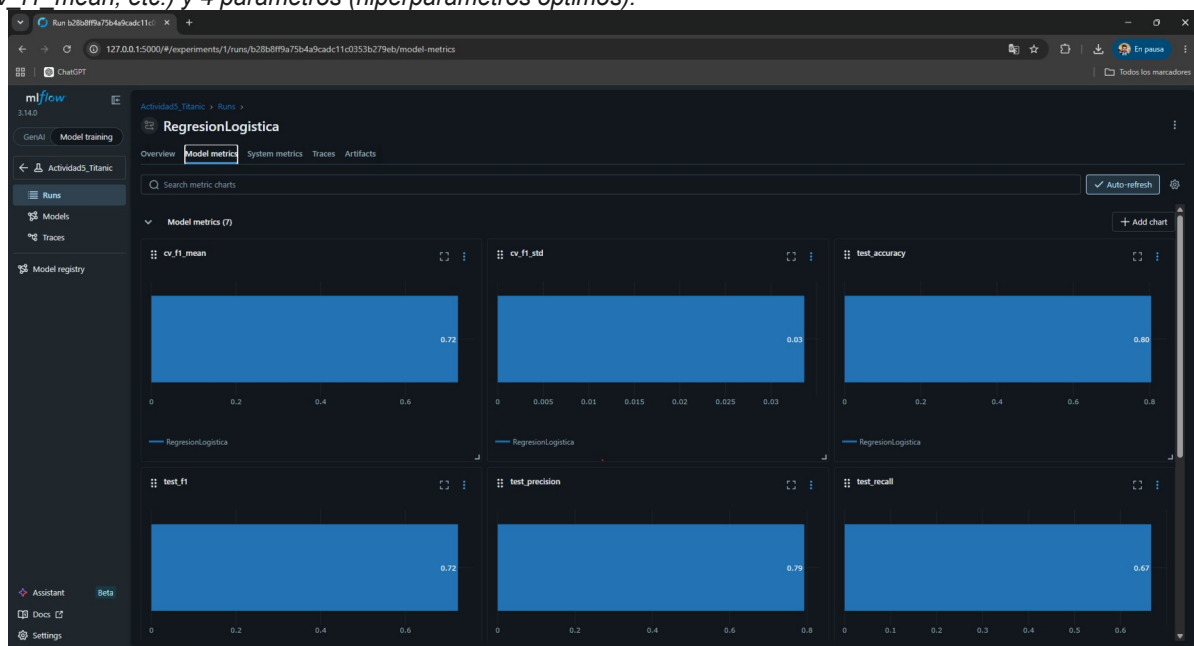


Figura 6. Graficas de metricas de la corrida RegresionLogistica en el panel de MLflow.

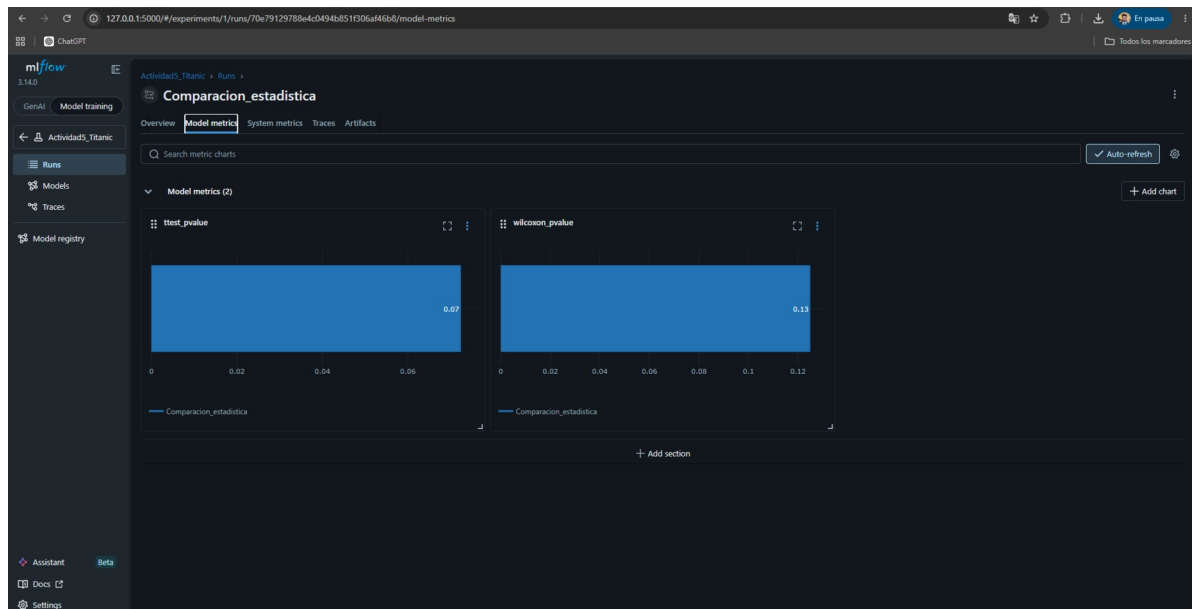


Figura 7. Corrida Comparacion_estadistica: valores p de la prueba t (0.07) y de Wilcoxon (0.13) registrados como metricas.

2.9 Reflexion sobre deuda tecnica y MLOps

El uso de MLflow mitiga la deuda de configuracion al centralizar parametros, metricas y artefactos, haciendo cada experimento trazable y reproducible (semilla fija, pipelines y versionado de datos). Frente a resultados equivalentes, se favorece la Regresion Logistica por su mayor interpretabilidad y menor costo de mantenimiento, decision alineada con las practicas de MLOps que buscan reducir la deuda tecnica a largo plazo.

3. Conclusiones

El proyecto demuestra un flujo completo de gestion de datos y MLOps: desde la obtencion y limpieza del dataset hasta el entrenamiento, ajuste de hiperparametros y registro reproducible de experimentos en MLflow. La calidad de los datos -imputacion de nulos, reglas de negocio, codificacion y escalado- resulto determinante para obtener modelos confiables.

Ambos modelos alcanzaron un desempeno similar (accuracy ~0.80, ROC-AUC ~0.84) sin diferencias estadisticamente significativas. Esto evidencia que, ante metricas comparables, la seleccion del modelo debe sustentarse tambien en criterios como interpretabilidad y mantenibilidad. Finalmente, la trazabilidad lograda con Git, el versionamiento de datos y MLflow constituye la base para escalar el proyecto hacia entornos productivos con menor deuda tecnica.

Referencias Bibliográficas

Belcic, I., y Stryker, C. (s.f.). What is hyperparameter tuning? IBM.

<https://www.ibm.com/think/topics/hyperparameter-tuning>

Caballar, R., y Stryker, C. (s.f.). Que es el rendimiento del modelo? IBM.

<https://www.ibm.com/mx-es/think/topics/model-performance>

Klingler, N. (2024). MLflow: Simplifying Machine Learning Experimentation.

<https://viso.ai/deep-learning/mlflow-machine-learning-experimentation/>

MLflow Project. (s.f.). Comprehensive experiment tracking. <https://mlflow.org/classical-ml/experiment-tracking>

Waskom, M. (s.f.). seaborn: statistical data visualization. <https://seaborn.pydata.org/>
